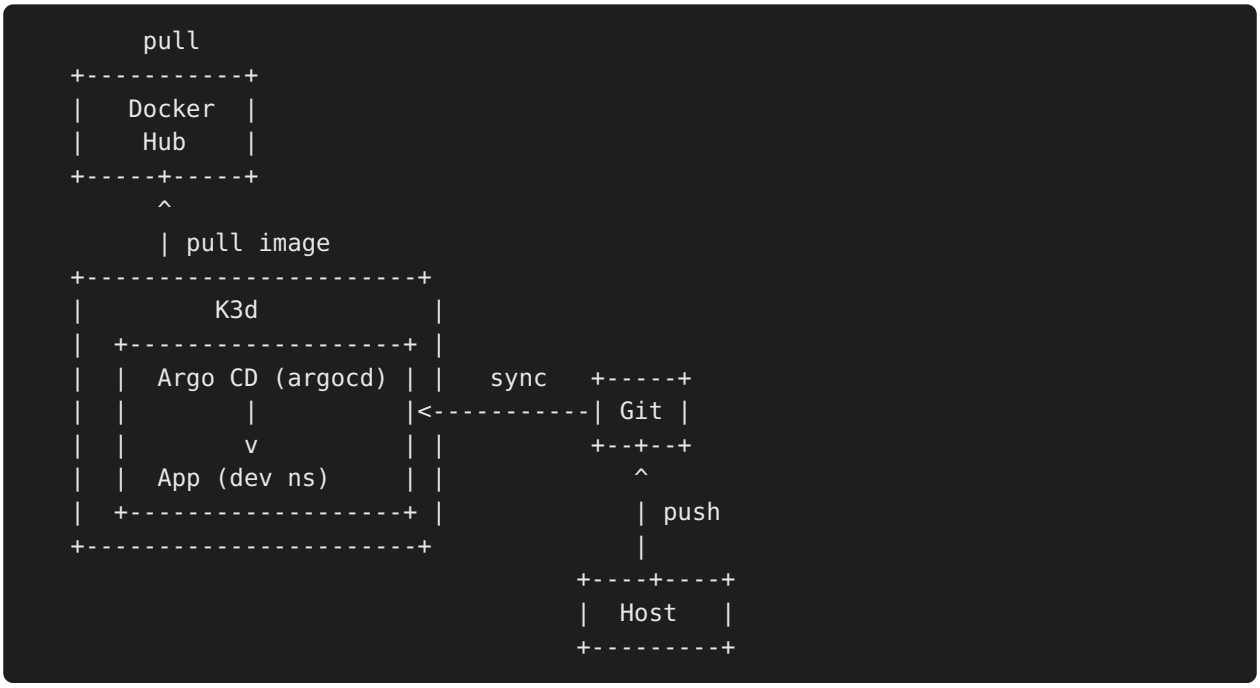
Part 3 — K3d and Argo CD — Step-by-Step Guide

This is a practical walkthrough of Part 3 of the subject, tailored to this repo's layout (login `oait-bou`, folders `p1/`, `p2/`, `p3/` with `scripts/` and `confs/` subfolders).

Key differences from Part 1 & 2: no Vagrant this time — everything runs on **one existing virtual machine**. You install K3d (not K3s directly — K3d runs K3s *inside Docker*), then set up Argo CD to continuously deploy an app from a public GitHub repo.

---

0. What you're building



- **K3d** = K3s packaged to run as Docker containers (simplifies setup vs. bare-metal K3s).
- **Argo CD** watches a Git repo and keeps your cluster's `dev` namespace in sync with it.
- You push a version change (`v1` → `v2`) to GitHub, and Argo CD auto-deploys it — no manual `kubectl apply`.

---

1. Prerequisites — one VM, no Vagrant

You can reuse an existing VM (e.g. your `p2` VM) or spin up a fresh one manually. It just needs Docker. Write everything needed as a **script** — you'll need to demo the install live during defense.

Create `p3/scripts/install.sh`:

```
#!/bin/bash
set -e

# 1. Docker (required by K3d)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# 2. kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# 3. K3d
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash

# 4. Argo CD CLI (optional but handy)
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argo
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
rm argocd-linux-amd64
```

> Log out/in (or `newgrp docker`) after the `usermod` so your shell picks up Docker group access.

---

2. Understand K3s vs K3d (read before you build)

- **K3s**: a real lightweight Kubernetes distro, installed directly on the host (what you did in Part 1/2).
- **K3d**: a thin wrapper that runs K3s **inside Docker containers**. Each "node" is a container. This is why Docker is a hard requirement here.

You do *not* need Vagrant/VMs-within-VMs for K3d — the containers themselves act as nodes.

---

3. Create the K3d cluster

```
k3d cluster create iot-p3 --api-port 6550 --servers 1 --agents 1 --port "8888:8888@loadbalancer"

# verify
kubectl get nodes
```

The `--port` flag maps the app's port (8888, used by Wil's sample app) from the K3d load balancer to your host, so `curl http://localhost:8888` works from outside the cluster.

---

4. Install Argo CD

Create the dedicated namespace and install Argo CD into it:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/in

# wait until everything is Running
kubectl get pods -n argocd -w
```

Access the Argo CD UI

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Get the initial admin password:

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
```

Open `https://localhost:8080` (user `admin`, the password above). You'll show this UI live during your defense — the subject explicitly says the Ingress from Part 2 is *not* enough, Argo CD's dashboard is what gets checked.

---

5. Create the `dev` namespace

```
kubectl create namespace dev
```

Recap of the two required namespaces:

Namespace	Purpose
`argocd`	Argo CD itself
`dev`	Your application, deployed **by** Argo CD

---

6. Pick your app: Wil's playground vs. your own

Option A — fastest, recommended for a first pass: Use Wil's pre-made image, already tagged with two versions:

- Image: `wil42/playground:v1` and `wil42/playground:v2`
- Port: `8888`
- `GET /` returns JSON like `{"status":"ok", "message":"v1"}`

Option B — build your own: Write a tiny app (e.g. a one-route HTTP server), push it to a **public** Docker Hub repo, and tag two visibly different versions `v1` and `v2` (e.g. a different response message).

Either way, the deliverable is the same: a `deployment.yaml` in your Git repo referencing the image.

---

7. Create your public GitHub repo

- Must be **public** (Argo CD needs to read it, and evaluators need to see it).
- Repo name must include your login somewhere, e.g. `oait-bou-iot-p3` or `oait-bou_argocd-app`.
- Push your Kubernetes manifests there (commonly under a `manifests/` folder).

Minimal `manifests/deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: playground
  namespace: dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: playground
  template:
    metadata:
      labels:
        app: playground
    spec:
      containers:
        - name: playground
          image: wil42/playground:v1
          ports:
            - containerPort: 8888
```

Add a matching `manifests/service.yaml` exposing port `8888` (type `NodePort` or `ClusterIP` + port-forward, or a `LoadBalancer` if you want the `k3d --port` mapping above to reach it directly).

Commit and push both files to your public repo.

---

8. Create the Argo CD Application

This is the piece that wires "my Git repo" → "my cluster's `dev` namespace" together.

`p3/confs/application.yaml`:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: playground
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<your-login>/<your-repo>.git
    targetRevision: HEAD
    path: manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Apply it:

```
kubectl apply -f p3/confs/application.yaml
```

`syncPolicy.automated` is what makes this **continuous deployment**: any push to your repo gets picked up and applied automatically (no manual `argocd app sync` needed, though you can still run it to force a sync).

---

9. Verify the deployment

```
kubectl get pods -n dev
kubectl get applications -n argocd   # Argo CD's view of your app: Healthy / Synced

curl http://localhost:8888/
# {"status":"ok", "message":"v1"}
```

Also check it in the Argo CD UI: app tile should show **Healthy** + **Synced**.

---

10. Demonstrate the v1 → v2 update (this is what gets tested live)

```
# locally, in your cloned repo
sed -i 's/playground:v1/playground:v2/' manifests/deployment.yaml
```

```
git add manifests/deployment.yaml
git commit -m "bump to v2"
git push
```

Wait a few seconds (or click **Refresh** / **Sync** in the Argo CD UI), then confirm:

```
kubectl get pods -n dev            # new pod, new image
curl http://localhost:8888/
# {"status":"ok", "message":"v2"}
```

Be ready to do this **live** during your defense, with whichever app you chose.

---

11. Folder structure checklist

```
p3/
├── scripts/
│   └── install.sh        # installs Docker, kubectl, k3d, argocd CLI
└── confs/
    └── application.yaml  # the Argo CD Application manifest
```

Your separate public GitHub repo holds the actual app manifests (`manifests/deployment.yaml`, `manifests/service.yaml`) — that repo is *referenced* by `application.yaml`, not copied into `p3/confs/`.

---

12. Common pitfalls

- **Forgetting `CreateNamespace=true`** → Argo CD fails to sync because `dev` doesn't exist yet (unless you created it manually in step 5, either works).
- **Private repo** → Argo CD can't clone it without extra credentials config; keep it public.
- **Wrong `path` in the Application spec** → must match the folder in your repo containing the manifests.
- **Docker not in the `docker` group** → `k3d cluster create` fails with permission denied on the socket.
- **Port collision** → if 8888 is already used on the host, change the `k3d cluster create --port` mapping.
- **Forgetting to re-tag/push the image** if you built your own app — Argo CD only sees what's in the **manifest** (the image tag string), not your local Docker builds.

---

13. Quick command reference

```
k3d cluster list
k3d cluster delete iot-p3          # teardown if you need to start over

kubectl get ns
kubectl get pods -n argocd
kubectl get pods -n dev
kubectl get applications -n argocd

argocd app get playground          # via Argo CD CLI, after `argocd login localhost:8080`
argocd app sync playground
```